

## Ex. 2.8 (22.5 pts) — Three Papers on Scheduling, Found by Search

*How I found them.* I searched for highly cited scheduling papers outside the course paper list whose full text is freely downloadable, then picked one from each of three eras — 1973 uniprocessor real-time theory, 2016 multicore kernel, 2019 datacentre microsecond-scale — so the three critiques would bite on different evidence rather than repeat one argument.

### Paper 1 — Liu and Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment” (JACM, 1973)

**Summary.** The paper asks how much of one processor can be committed to  $m$  periodic tasks while still guaranteeing every deadline. A task is reduced to two numbers, its period  $T_i$  and its run-time  $C_i$ , under five assumptions: periodic requests, a deadline equal to the next request, independent tasks, constant run-times, and aperiodic work confined to initialisation and failure recovery.

For fixed priorities, Theorem 1 identifies the *critical instant* — “A critical instant for any task occurs whenever the task is requested simultaneously with requests for all higher priority tasks” — which turns an infinite schedule into a finite check. Theorem 2 shows the *rate-monotonic* rule (shorter period, higher priority, regardless of run-time) is optimal among all fixed-priority rules, and Theorem 5 bounds utilisation at  $U = m(2^{1/m} - 1)$ : about 0.83 for two tasks, 0.78 for three, falling to  $\ln 2 \approx 0.69$ . Section 7 instead assigns priorities by nearest deadline, and Theorem 7 proves that *deadline driven* rule feasible if and only if  $\sum_i C_i/T_i \leq 1$ , so the same hardware is schedulable at 100% load. Section 8 mixes the two, running the  $k$  shortest-period tasks rate-monotonically on the interrupt hardware; the worked example in Section 9,  $T = (3, 4, 5)$  with  $C_1 = C_2 = 1$ , reaches 98.3% mixed against 100% deadline-driven and 78.3% rate-monotonic.

**Critical judgment.** The cost model is the weak point. Assumption (A4) folds context-switch cost into the constant  $C_i$ , claiming “the bookkeeping time necessary to request a successor and the costs of preemptions can be taken into account”. It cannot: how often a task is preempted depends on the whole task set, not on the task, and rate-monotonic scheduling preempts far more than deadline-driven scheduling does. The practical case for the fixed-priority algorithm is exactly the quantity the model prices at zero. Section 6 concedes that “the practical costs of switching between tasks must still be counted”, and then never counts them.

The more consequential gap is independence. The conclusion is candid about the other assumptions, calling periodicity and constant run-times “the most important and least defensible” and admitting that without periodicity “None of our analytic work would remain valid”; it even reopens the deadline assumption. Independence alone is never reopened, yet it is what real systems break first: tasks share mutexes, and a low-priority task holding a lock blocks a high-priority one for a time the utilisation bound does not model. That hole stayed open until Sha, Rajkumar and Lehoczky’s priority-inheritance protocols added an explicit blocking term to the test; the 1997 Mars Pathfinder resets showed what it costs in the field. The framework itself won completely — but the paper’s parting advice, to make periodicity and constant run-times “a design goal”, is the part practice discarded, in favour of sporadic-task and server models.

### Paper 2 — Lozi, Lepers, Funston, Gaud, Quéma and Fedorova, “The Linux Scheduler: a Decade of Wasted Cores” (EuroSys, 2016)

**Summary.** The paper takes one invariant — “make sure that ready threads are scheduled on available cores” — and shows Linux’s Completely Fair Scheduler violating it for seconds at a time.

The cause is structural: per-core runqueues force periodic load balancing, balancing is too expensive to run often, and the load metric (weight scaled by utilisation, then divided by the cgroup’s thread count) is complicated enough to hide imbalance.

Four bugs follow. *Group Imbalance*: a stealing core compares the *average* load of a scheduling group, so one heavy thread masks idle cores beside it; comparing minimum load instead made a kernel `make` 13% faster and NAS `1u` 13× faster. *Scheduling Group Construction*: groups are built from core 0’s viewpoint, so nodes two hops apart land in both of the groups  $\{0, 1, 2, 4, 6\}$  and  $\{1, 2, 3, 4, 5, 7\}$ , their averages always match and no work moves — `1u` takes 1040s instead of 38s. *Overload-on-Wakeup*: for cache reuse a waking thread is placed only among cores of the waker’s node, so it can wake onto a busy core while others idle; the fix cut TPC-H query 18 by 22.2% and the full benchmark by 13.2%. *Missing Scheduling Domains*: a call dropped during refactoring means disabling and re-enabling any core silently stops all cross-NUMA balancing, and `1u` runs 137.59× slower. None of these crash anything, so the authors add an online sanity checker for the invariant and a trace visualiser, the latter needing under 150 lines of kernel instrumentation.

**Critical judgment.** The bug analyses are exact, but the evidence is wide in workloads and narrow in hardware: every measurement comes from one 8-node AMD Bulldozer server. Table 4 nevertheless records “Impacted applications: All” for three of the four bugs, two of them topology-dependent — Scheduling Group Construction needs nodes two hops apart — so a claim about which *applications* suffer is doing duty as one about which *machines* do, on a sample of one.

The energy claim is weaker. The introduction asserts that “Energy waste is proportional” to the lost performance, yet power runs through the paper as a design pressure without ever being measured — no power figure, axis or table — and it bites on the Overload-on-Wakeup fix. The authors raise that tradeoff themselves rather than hiding it, since waking a thread on a long-idle core forces it out of a low-power state, but their answer is to “only enforce the new wakeup strategy if the system’s power management policy does not allow cores to enter low-power states at all”. The fix that wins most on their database workload is therefore disabled on virtually every production server, and the energy argument that might justify re-enabling it is asserted rather than measured.

History has been kind to both forecasts. “New scheduler designs come and go” held: CFS was itself replaced, by EEVDF, in kernel 6.6 (2023). The modular alternative of Section 5 — “a scheduler that is a collection of modules: the core module and optimization modules” — was then taken up in spirit by `sched_ext` in 6.12 (2024), which moves policy into BPF programs outside the core. The fit is loose, since `sched_ext` replaces the whole scheduler rather than advising an invariant-keeping core, but the instinct is the paper’s.

### Paper 3 — Kaffes, Chong, Humphries, Belay, Mazières and Kozyrakis, “Shinjuku: Preemptive Scheduling for $\mu$ second-scale Tail Latency” (NSDI, 2019)

**Summary.** Dataplane operating systems reach microsecond latencies by never preempting: IX hashes requests to per-core queues (distributed FCFS), and ZygOS work-steals to approximate a central FCFS queue. Both let a short request queue behind a long one, which is fatal when service times are heavy-tailed or bimodal, where theory prefers processor sharing — and processor sharing needs preemption. Linux cannot supply it, since CFS targets a 6ms preemption latency with a 0.75ms minimum.

Shinjuku is a single-address-space OS on Dune in which one dispatcher thread owns the queues and interrupts workers directly. Removing both VM exits from the interrupt path — posted interrupts on the receiver, a directly mapped APIC on the sender — takes the sender from 2081 to 298 cycles and the receiver from 2662 to 1212, and context switches drop to 36–109 cycles; at a

5  $\mu$ s quantum that costs under 10 % of worker throughput. Two policies exploit it: a single queue preempting every 5–15  $\mu$ s, and a multi-queue policy serving whichever queue’s head request has the largest ratio of queueing time to its SLO target. Against ZygOS, Shinjuku obtains roughly 50 % lower tail latency and 5 $\times$  the throughput at a 300  $\mu$ s target on a bimodal synthetic workload, and on RocksDB serving 99.5 % GET plus 0.5 % SCAN, 88 % lower tail latency with 6.6 $\times$  the throughput.

**Critical judgment.** Centralisation is what makes the policies possible and is the claim least tested. Section 3.6 admits one dispatcher saturates at about a socket — near-linear to 11 workers and 5 M requests/s, 9.5 M with two — and proposes to scale further by “steer[ing] requests to different dispatchers using the NIC RSS feature”. That is the distributed queueing Section 2 blames for head-of-line blocking, reintroduced at the granularity of dispatcher groups, and what it costs is never measured: the only multi-dispatcher experiment issues “short requests with 1  $\mu$ sec fixed service time to stress the dispatcher”, the one distribution under which head-of-line blocking cannot occur. The headline claim is about heavy tails; the scaling result is measured where tails are absent.

The security discussion is honest, but its remedy is still a promise. Mapping APIC registers into the process means, as the authors say, that “one process could launch a denial-of-service attack on another process by issuing a large number of interrupts to a specific core”; they answer that “future versions of Shinjuku will run application code in ring 3 ... eliminating this attack vector with very small overhead”, citing an 84-cycle ring transition. But every evaluation number was taken with applications in ring 0, chosen “to avoid the address space crossings” — crossings that, at a 5  $\mu$ s quantum, would recur on every preemption. The system that was measured is not the system that would be deployable.

The premise — that the obstacle was mechanism rather than policy — has held up: the hardware the paper asks for, “support for lightweight user-level interrupts”, later shipped as Intel’s user interrupts (UINTR) in Sapphire Rapids.

## References

- [1] C. L. Liu and J. W. Layland. “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment.” *Journal of the ACM*, vol. 20, no. 1, January 1973, pp. 46–61. DOI: 10.1145/321738.321743.
- [2] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma and A. Fedorova. “The Linux Scheduler: a Decade of Wasted Cores.” *Proc. 11th European Conference on Computer Systems (EuroSys ’16)*, London, April 2016. DOI: 10.1145/2901318.2901326.
- [3] K. Kaffes, T. Chong, J. T. Humphries, A. Belay, D. Mazières and C. Kozyrakis. “Shinjuku: Preemptive Scheduling for  $\mu$ second-scale Tail Latency.” *Proc. 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’19)*, Boston, February 2019, pp. 345–360. <https://www.usenix.org/conference/nsdi19/presentation/kaffes>
- [4] L. Sha, R. Rajkumar and J. P. Lehoczky. “Priority Inheritance Protocols: An Approach to Real-Time Synchronization.” *IEEE Transactions on Computers*, vol. 39, no. 9, September 1990, pp. 1175–1185. (Cited above for the blocking term missing from [1].)